OWASP Top 10 for LLM Applications

Voight Alignment (v2.0)

Voight

May 2026

Contents

Document Control

Field	Value
Document Title	OWASP Top 10 for LLM Applications — Voight Alignment
Document Version	1.0
Framework Tracked	OWASP Top 10 for LLM Applications, v2.0
Status	Published
Issue Date	May 26, 2026
Owner	Voight — Security Office
Approver	Founding Team
Classification	Public
Next Review	November 26, 2026

This document tracks the OWASP Top 10 for LLM Applications 2025 (v2.0), the current published edition. It will be re-versioned when OWASP publishes its next edition (2026 cycle in community survey at the time of writing).

This document describes how Voight (“Voight”, “we”, “us”, “our”) relates to the OWASP Top 10 for LLM Applications. It is written for security reviewers, AI engineers, and procurement teams evaluating Voight, as well as for Voight customers who use the platform to harden their own LLM applications.

1. Executive Summary

The OWASP Top 10 for LLM Applications is the industry reference for the most critical security risks in software that uses large language models. Voight relates to that list from two distinct angles, and this document addresses both honestly.

Part 1 — Voight's own posture. Today, Voight does not operate a large language model within its own product. Voight is the observability platform that *receives* telemetry from LLM applications; it is not itself an LLM application in production. Part 1 therefore documents the security of the platform that handles that telemetry, and sets out a forward commitment for the LLM-powered features Voight has on its roadmap.

Part 2 — How Voight helps you. Because Voight is purpose-built to observe LLM applications, it is a natural detection and monitoring layer for many of the risks on the list. Part 2 maps each of the ten risks to the concrete Voight capabilities that help a customer detect, investigate, or constrain that risk in their own application — and is equally clear about the risks where an observability tool offers little or nothing.

Two honest framing points run through the entire document:

1. ***Observability is a detection and monitoring discipline, not a prevention control.*** For most of the ten risks, Voight helps you *see* and *investigate* a problem; it does not sit in the request path to *block* it. Where this distinction matters, we state it explicitly.
2. ***We do not claim coverage we do not have.*** Two of the ten risks — Data and Model Poisoning (LLM04) and Vector and Embedding Weaknesses (LLM08) — fall largely outside what an observability platform addresses, because Voight neither trains models nor manages vector stores. We say so plainly rather than stretch a weak angle.

A coverage summary appears in §7, and a consolidated statement of what Voight does *not* do appears in §8.

2. The OWASP Top 10 for LLM Applications (v2.0)

For reference, the ten risks tracked by this document are:

Code	Risk
LLM01	Prompt Injection
LLM02	Sensitive Information Disclosure
LLM03	Supply Chain
LLM04	Data and Model Poisoning
LLM05	Improper Output Handling
LLM06	Excessive Agency
LLM07	System Prompt Leakage
LLM08	Vector and Embedding Weaknesses
LLM09	Misinformation
LLM10	Unbounded Consumption

The authoritative definitions are maintained by the OWASP GenAI Security Project at `genai.owasp.org/llm-top-10`. This document does not restate them in full; it summarises each risk in one or two sentences and then focuses on Voight's relationship to it.

3. Scope and Framing

3.1 Two Perspectives

This document deliberately separates two questions that are often conflated:

- ***"Is Voight itself secure against these risks?"*** — addressed in Part 1 (§4).
- ***"Does Voight help me secure my application against these risks?"*** — addressed in Part 2 (§5–§6).

Keeping them apart matters. A reader evaluating Voight as a vendor cares about Part 1. A reader evaluating Voight as a security tool cares about Part 2. Conflating them produces a document that overstates Voight's own exposure and understates its value, or the reverse.

3.2 What Observability Can and Cannot Do

Voight is an observability platform. It captures, stores, and presents telemetry about LLM calls — model, tokens, latency, tool calls, cost, outcomes, and (subject to the customer's privacy settings) prompt and completion content. This shapes what Voight can contribute to each risk:

- ***It can*** make an attack or failure *visible* — after the fact, or in near-real-time through alerts.
- ***It can*** preserve the *evidence* needed to investigate an incident, attribute it to a model, agent, or user, and measure its blast radius.
- ***It can*** surface *patterns* — cost spikes, error surges, anomalous tool usage — that indicate a risk is materialising.
- ***It cannot*** sit inline and *block* a malicious prompt, sanitise an output, or veto a tool call. Voight observes; it does not gate.

Throughout Part 2, "Voight helps" should be read in this sense: detection, investigation, and monitoring — the layer that tells you a control failed, not the control itself.

4. Part 1 — Voight's Own Security Posture

4.1 Is Voight an LLM Application Today?

No. As of the issue date of this document, Voight does not call a large language model within its own production product. The Voight platform ingests telemetry generated by *the customer's* LLM applications and presents it in a dashboard. The "AI Apps" section of the product is named for the customer applications it observes — not for any use of an LLM by Voight itself.

Consequently, the risks on the OWASP list that are intrinsic to *operating* an LLM — for example, an attacker injecting a prompt into Voight's own model, or Voight's own model being poisoned — do not currently apply to Voight, because there is no such model in the product.

4.2 Platform Security Baseline

What Voight does operate is a conventional web platform — an ingest API, a database, and a dashboard. Its security is documented in full in Voight's GDPR Compliance Documentation (§8, "Security Measures") and is summarised here:

Control	Implementation
Transport security	TLS 1.3 on all public endpoints
Storage encryption	AES-256 at rest; secrets stored hashed
Authentication	Privy-brokered identity for users; hashed, revocable `vk_` keys for ingest
Access control	Least-privilege; production data access restricted and logged
Supply chain	Dependabot monitoring; npm packages published with provenance attestations
Rate limiting	Per-tier burst limits on the ingest API
Logging	90-day retention; personal data minimised in logs

These controls are the foundation on which any future LLM feature will be built.

4.3 Planned LLM Features and Forward Commitment

Voight has LLM-powered features on its roadmap, including:

- ***Smart Trace*** — LLM-assisted summarisation and explanation of a trace.
- ***Prompt Scorer*** — automated quality and risk scoring of prompts.

- ***Debug Agent*** — an agent that helps diagnose failures in a customer's traces.

When these ship, Voight will itself become an LLM application and will fall within the scope of this Top 10 as an operator, not only as a tool. Voight makes the following commitment:

> When any LLM-powered feature reaches production, Voight will hold that feature to the same controls this document describes for our customers, and will re-version this document to assess the feature against each relevant risk — in particular Prompt Injection (LLM01), Sensitive Information Disclosure (LLM02), System Prompt Leakage (LLM07), Excessive Agency (LLM06), and Unbounded Consumption (LLM10) — before the feature is generally available.

This is the same forward-commitment model Voight applies to planned sub-processors in its GDPR documentation: declare the change before it happens, and update the public record when it does.

5. Part 2 — How Voight Helps You Address the Top 10

Each risk below follows the same structure: a short statement of the risk, how Voight helps, and an honest note on the limits of that help.

LLM01 — Prompt Injection

The risk. Adversarial input — direct, or indirect via content the model retrieves — alters the model's behaviour, causing it to ignore instructions, exfiltrate data, or misuse tools.

How Voight helps. Voight captures the full request context for each LLM call: the prompts (subject to your privacy level), the model's response, the tools it then invoked, and the outcome. When an injection succeeds, this trace is the forensic record — it shows what the model was told, what it did differently, and which downstream tools or data were touched. With `withTrace`, a multi-step agent run is reconstructed end-to-end, so an indirect injection that surfaces three steps later can be traced back to the content that introduced it. Alerts on error-rate and anomalous tool usage can flag injection *patterns* in near-real-time.

Limits. Voight is post-hoc and observational. It does not inspect or filter prompts inline, and it does not block an injection in progress. It tells you an injection happened and gives you the evidence to understand it; preventing it requires input-validation controls in your own application.

LLM02 — Sensitive Information Disclosure

The risk. Confidential data — PII, secrets, proprietary content — is exposed through the model's output or through the surrounding application, including through the telemetry that observability tools collect.

How Voight helps. This is Voight's strongest alignment, because Voight is designed not to *become* a disclosure vector itself. The SDK runs one of three privacy levels ***locally, inside your own process, before any payload leaves it***:

- ***Minimal*** — free-text content is dropped entirely; only structural metadata is sent.
- ***Standard*** (default) — every string is passed through `scrubPii()`, a catalogue of thirteen patterns plus a Luhn-validated credit-card check, replacing secrets and identifiers with tagged tokens (`[REDACTED-API-KEY]`, `[REDACTED-EMAIL]`, …).
- ***Full*** — no SDK-side filtering, for environments where the customer has already scrubbed upstream.

Every event carries a `privacyLevel` stamp, giving you an audit trail that scrubbing occurred. Because the filter runs before transmission, sensitive data never reaches Voight in the first place under Minimal or Standard.

Limits. Standard mode scrubs known patterns; it is conservative by design and will not catch every bespoke secret format. It reduces disclosure risk in your *telemetry*; it does not scrub the live output your application returns to its own end users.

LLM03 — Supply Chain

The risk. Compromised models, datasets, plugins, or dependencies introduce vulnerabilities into the LLM application.

How Voight helps. Voight records the model identifier and provider for every call, per event, via `metadata.source`. This gives you an accurate, continuously updated inventory of which models and providers your application actually uses in production — the foundation of any supply-chain assessment. A sudden appearance of an unexpected model or provider is visible immediately. On its own side, Voight hardens its supply chain with Dependabot monitoring and npm provenance attestations on its published packages, so the SDK you install is verifiable.

Limits. Voight inventories the models you call; it does not attest to the integrity of those upstream models or scan third-party plugins. Vendor due diligence on your model providers remains your responsibility.

LLM04 — Data and Model Poisoning

The risk. Training, fine-tuning, or embedding data is tampered with to introduce backdoors, biases, or vulnerabilities into the model.

How Voight helps. Limited and indirect. Voight does not train, fine-tune, or host models, so it has no role in the integrity of the training pipeline where this risk lives. The only adjacent contribution is detective: if a poisoned model produces anomalous outputs or error patterns at runtime, Voight's monitoring may surface that anomaly — but this is a weak, downstream signal, not a control against poisoning.

Limits. This risk falls outside the scope of an observability platform. Mitigating it requires controls at the training-data and model-provenance layer, which Voight does not operate.

LLM05 — Improper Output Handling

The risk. Output from the model is passed to downstream components without sufficient validation or sanitisation, enabling injection, code execution, or data corruption in those components.

How Voight helps. Voight captures the model's raw output, the outcome of the call, and the error chain when a downstream component rejects or mishandles that output. When improper output handling causes a failure, the trace shows the exact output that triggered it and where it broke. Error-rate alerts surface a rise in handling failures before they become widespread.

Limits. Voight records what the output was and what happened next; it does not validate or sanitise the output itself. The sanitisation control must live in your application between the model and the downstream component.

LLM06 — Excessive Agency

The risk. An LLM-based system is granted more autonomy, permissions, or tool access than it needs, so that a manipulated model can take damaging actions.

How Voight helps. Voight maintains a per-agent, per-trace audit trail of every tool the model invoked, how often, with what success rate, and at what latency (the Tools view). This is direct evidence of the *actual* agency your system exercises, as opposed to the agency you intended. Reviewing it reveals tools being called more than expected, tools that should never have been reachable, or an agent escalating its behaviour — the signatures of excessive agency. Orphan-spike alerts can flag tool activity that has detached from its expected trace.

Limits. Voight shows you what the agent did; it does not enforce permission boundaries. Constraining agency — least-privilege tool design, human-in-the-loop approval — is an architectural control in your application.

LLM07 — System Prompt Leakage

The risk. The system prompt — which may contain instructions, credentials, or business logic the developer assumed was hidden — is exposed to users.

How Voight helps. Two ways. First, your privacy level governs whether prompt content is captured at all, so you control whether system prompts even enter your telemetry. Second, `scrubPii()` in Standard mode redacts secrets embedded in captured prompts, so a credential mistakenly placed in a system prompt is tokenised rather than stored in clear text. If a leak occurs, the trace record helps you identify which prompt was exposed and through which call.

Limits. Voight protects system-prompt material *within its own telemetry*; it does not prevent your application from returning the system prompt to an end user. The architectural fix — keeping secrets out of prompts entirely — is yours to implement.

LLM08 — Vector and Embedding Weaknesses

The risk. Weaknesses in how vectors and embeddings are generated, stored, or retrieved — in RAG pipelines and semantic search — allow data leakage, poisoning, or manipulation.

How Voight helps. Limited. Voight does not generate, store, or query embeddings, and it does not operate a vector database, so it has no direct visibility into this layer. If your RAG pipeline's retrieval and generation steps are instrumented with `withTrace`, Voight can show

the LLM calls that consumed the retrieved context — but it does not see the vector store itself, the retrieval scoring, or the embedding generation.

Limits. This risk lives in infrastructure Voight does not touch. Securing your vector store, retrieval logic, and embedding pipeline is outside what Voight observes.

LLM09 — Misinformation

The risk. The model produces false or misleading output that users over-rely on, leading to harm or poor decisions.

How Voight helps. Voight gives you the data to *monitor* the conditions associated with misinformation: which models are in use, error and outcome rates, and (with custom instrumentation via `log()` inside `withTrace`) domain events such as "retrieval returned zero results" — a common precursor to hallucinated answers. Cost and error alerts can flag a model behaving abnormally.

Limits. Misinformation is fundamentally a content-quality problem. Voight does not evaluate the factual accuracy of outputs; detecting misinformation reliably requires evaluation tooling and ground-truth checks at the application layer. Voight's contribution here is supporting signal, not detection of falsehood itself.

LLM10 — Unbounded Consumption

The risk. An LLM application operates without resource constraints, allowing denial-of-service, runaway cost, or model-extraction through excessive querying.

How Voight helps. This is one of Voight's strongest alignments. Voight tracks token counts (input, output, cache reads, cache creations), cost in USD, and latency for every call, aggregated by model, agent, and end user. Cost-spike and event-surge alerts fire when consumption departs from its baseline — the earliest practical signal of a denial-of-wallet attack or a runaway loop. Per-user spend attribution identifies *which* caller is driving consumption. Voight's own ingest API is rate-limited per tier, constraining its own exposure.

Limits. Voight measures and alerts on consumption; it does not itself throttle your model calls. Enforcing quotas and rate limits on your LLM usage is a control in your application or gateway. Voight tells you, quickly, when consumption is abnormal.

6. Coverage Summary

The table below summarises Voight's alignment with each risk. "Detect / monitor" is the dominant mode throughout — consistent with §3.2.

Code	Risk	Voight alignment	Primary mechanism
LLM01	Prompt Injection	Strong (detective)	Full trace forensics + tool-usage alerts
LLM02	Sensitive Information Disclosure	***Strongest***	Local 3-level scrubbing before transmission
LLM03	Supply Chain	Moderate	Per-event model/provider inventory + own provenance
LLM04	Data and Model Poisoning	Limited	No role; weak downstream anomaly signal only
LLM05	Improper Output Handling	Strong (detective)	Output + outcome + error-chain capture
LLM06	Excessive Agency	Strong (detective)	Per-agent tool-call audit trail
LLM07	System Prompt Leakage	Strong (preventive on telemetry)	Privacy levels + prompt scrubbing
LLM08	Vector and Embedding Weaknesses	Limited	Outside observed infrastructure
LLM09	Misinformation	Moderate	Supporting signals; no factuality check
LLM10	Unbounded Consumption	***Strong***	Token/cost tracking + spike alerts + per-user spend

7. What Voight Does Not Do

In the interest of an honest record, Voight explicitly does ***not*** provide the following, and customers should not rely on it for them:

- ***Inline prevention.*** Voight does not sit in the request path. It does not block prompts, sanitise outputs, or veto tool calls. It is a detection and monitoring layer.
- ***Model training integrity.*** Voight does not train, fine-tune, or host models and offers no control over training-data or model poisoning (LLM04).
- ***Vector store security.*** Voight does not operate or inspect vector databases, embedding pipelines, or retrieval scoring (LLM08).
- ***Factuality evaluation.*** Voight does not assess whether an output is true. It does not detect misinformation on its own (LLM09).
- ***Quota enforcement on your models.*** Voight measures and alerts on consumption but does not throttle your LLM calls (LLM10).
- ***Guaranteed exhaustive PII detection.*** Standard-mode scrubbing is conservative and pattern-based; it reduces, but does not eliminate, the possibility of an unrecognised secret format reaching Voight (LLM02).

Stating these boundaries is deliberate. An observability tool that claims to prevent every risk on this list would be misrepresenting what observability is.

8. Governance and References

Field	Value
Framework	OWASP Top 10 for LLM Applications, v2.0
Authoritative source	genai.owasp.org/llm-top-10
Security contact	team@voight.xyz
Related document	Voight GDPR Compliance Documentation (docs.voight.xyz/trust/gdpr)
Review cycle	Every six months, and on publication of a new OWASP edition

Voight commits to re-versioning this document when OWASP publishes its next edition of the Top 10 for LLM Applications, and when any Voight LLM-powered feature (§4.3) reaches production.

Annex A — Feature-to-Risk Mapping

A reverse index: each Voight capability and the risks it contributes to.

Voight capability	Risks supported
Three-level local PII scrubbing (`scrubPii`, privacy levels)	LLM02, LLM07
`privacyLevel` per-event stamp (audit trail)	LLM02, LLM07
Full trace capture (`withTrace`)	LLM01, LLM05, LLM06, LLM09
Per-event model/provider attribution (`metadata.source`)	LLM03
Tool-call audit trail (Tools view)	LLM06
Token, cost, and latency tracking	LLM10
Per-user spend attribution	LLM10
Alerts: cost spike, event surge, error rate, orphan spike	LLM01, LLM05, LLM09, LLM10
Outcome and error-chain capture	LLM05
npm provenance + Dependabot (own supply chain)	LLM03
Per-tier ingest rate limiting (own platform)	LLM10

Annex B — What Changed in v2.0

For readers familiar with the 2023 edition, the v2.0 list reorganised several entries. This annex records the mapping so the alignment above is unambiguous.

v2.0	Title	Relative to 2023 edition
LLM01	Prompt Injection	Unchanged
LLM02	Sensitive Information Disclosure	Risen in rank (was LLM06)
LLM03	Supply Chain	Broadened from “Supply Chain Vulnerabilities”
LLM04	Data and Model Poisoning	Was “Training Data Poisoning” (LLM03)
LLM05	Improper Output Handling	Was “Insecure Output Handling” (LLM02)
LLM06	Excessive Agency	Risen in rank (was LLM08)
LLM07	System Prompt Leakage	New in v2.0
LLM08	Vector and Embedding Weaknesses	New in v2.0
LLM09	Misinformation	Replaced “Overreliance”
LLM10	Unbounded Consumption	Broadened from “Model Denial of Service”

A separate emerging project, the ***OWASP Top 10 for Agentic Applications (2026)***, addresses risks specific to agentic systems with tool access and multi-step autonomy. Given Voight’s focus on agent observability, Voight is monitoring that project and may publish a dedicated alignment document when it is finalised.

Annex C — Document Version History

Version	Date	Change	Author
1.0	2026-05-26	Initial publication against OWASP LLM Top 10 v2.0.	Voight — Security Office

Contact

For any question about this document or about how Voight supports the security of LLM applications:

Voight — Security Office

team@voight.xyz

voight.xyz/trust · docs.voight.xyz/trust
